

Assignment – 11_5

Data Structures with AI: Implementing Fundamental Structures

Name: Rishitha Gangidi

HT.NO: 2303A52149

Batch: 35

Task Description - 1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

```
class Stack:
```

```
    pass
```

Expected Output:

- A functional stack implementation with all required methods and docstrings.

```
1  class Stack:
2      def __init__(self):
3          self._items = []
4
5      def push(self, item):
6          """Push an item onto the stack."""
7          self._items.append(item)
8
9      def pop(self):
10         """Pop the top item from the stack and return it."""
11         if self.is_empty():
12             raise IndexError("pop from empty stack")
13         return self._items.pop()
14
15     def peek(self):
16         """Return the top item without removing it."""
17         if self.is_empty():
18             raise IndexError("peek from empty stack")
19         return self._items[-1]
20
21     def is_empty(self):
22         """Check if the stack is empty."""
23         return len(self._items) == 0
24
```

Task Description - 2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
```

```
    pass
```

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

```
24
25 class Queue:
26     def __init__(self):
27         self._items = []
28
29     def enqueue(self, item):
30         """Add an item to the end of the queue."""
31         self._items.append(item)
32
33     def dequeue(self):
34         """Remove and return the front item from the queue."""
35         if self.is_empty():
36             raise IndexError("dequeue from empty queue")
37         return self._items.pop(0)
38
39     def peek(self):
40         """Return the front item without removing it."""
41         if self.is_empty():
42             raise IndexError("peek from empty queue")
43         return self._items[0]
44
45     def size(self):
46         """Return the number of items in the queue."""
47         return len(self._items)
48
49     def is_empty(self):
50         """Check if the queue is empty."""
51         return len(self._items) == 0
```

Task Description - 3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

```
class Node:
```

```
    pass
```

```
class LinkedList:
```

```
    pass
```

Expected Output:

- A working linked list implementation with clear method documentation.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        """Insert a new node with the given data at the end of the list."""
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node

    def display(self):
        """Display the elements of the linked list."""
        current = self.head
        while current:
            print(current.data, end=" -> ")
            current = current.next
        print("None")
```

Task Description - 4 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

```
class HashTable:
    pass
```

Expected Output:

- Collision handling using chaining, with well-commented methods.

```

class HashTable:
    def __init__(self, size=10):
        """
        Initialize the hash table with a given size.
        Uses a list of lists for chaining to handle collisions.
        """
        self.size = size
        self.table = [[] for _ in range(size)] # List of buckets, each bucket is a List for chaining

    def _hash(self, key):
        """
        Simple hash function using built-in hash and modulo.
        """
        return hash(key) % self.size

    def insert(self, key, value):
        """
        Insert a key-value pair into the hash table.
        If the key already exists, update the value.
        """
        index = self._hash(key)
        bucket = self.table[index]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value) # Update existing

```

```

class HashTable:
    def insert(self, key, value):
        """
        Insert a key-value pair into the hash table.
        If the key already exists, update the value.
        """
        index = self._hash(key)
        bucket = self.table[index]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value) # Update existing
        bucket.append((key, value)) # Add new

    def search(self, key):
        """
        Search for a key in the hash table and return its value if found.
        Returns None if not found.
        """
        index = self._hash(key)
        bucket = self.table[index]
        for k, v in bucket:
            if k == key:
                return v
        return None

    def delete(self, key):
        """
        Delete a key-value pair from the hash table.
        Returns True if deleted, False if key not found.
        """
        index = self._hash(key)
        bucket = self.table[index]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                del bucket[i]
                return True
        return False

```

Task Description - 5 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

```

class Graph:
    pass

```

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

```

167 class Graph:
168     def __init__(self):
169         """
170         Initialize an empty graph using an adjacency list.
171         The adjacency list is a dictionary where keys are vertices and values are lists of connected
172         """
173         self.adj_list = {}
174
175     def add_vertex(self, vertex):
176         """
177         Add a vertex to the graph if it doesn't already exist.
178         """
179         if vertex not in self.adj_list:
180             self.adj_list[vertex] = []
181
182     def add_edge(self, vertex1, vertex2):
183         """
184         Add an undirected edge between vertex1 and vertex2.
185         Adds both vertices if they don't exist.
186         """
187         if vertex1 not in self.adj_list:
188             self.add_vertex(vertex1)
189         if vertex2 not in self.adj_list:
190             self.add_vertex(vertex2)
191         # Assuming undirected graph
192         if vertex2 not in self.adj_list[vertex1]:
193             self.adj_list[vertex1].append(vertex2)
194         if vertex1 not in self.adj_list[vertex2]:
195             self.adj_list[vertex2].append(vertex1)
196
197     def display(self):
198         """
199         Display the adjacency list of the graph.
200         """
201         for vertex, neighbors in self.adj_list.items():
202             print(f"{vertex}: {neighbors}")
203
204

```

Task Description - 6: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

- For each feature, select the most appropriate data structure from

the list below:

- o Stack
- o Queue
- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```
205
206 import heapq
207
208 class EmergencyQueue:
209     def __init__(self):
210         """
211         Initialize the emergency queue using a min-heap for priority management.
212         Each entry is a tuple: (priority, patient_id, patient_name)
213         Lower priority number means higher urgency.
214         """
215         self._queue = []
216         self._counter = 0 # To handle ties in priority
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

```
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code\Code -n.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/AI Assistant Coding/vscode/Assignment_11(5).py"
Added patient Alice (ID: P001) with priority 1.
Added patient Bob (ID: P002) with priority 3.
Added patient Charlie (ID: P003) with priority 2.
Current emergency queue (priority, ID, Name):
  Priority 1: Alice (ID: P001)
  Priority 2: Charlie (ID: P003)
  Priority 3: Bob (ID: P002)
Treating patient Alice (ID: P001) with priority 1.
Treating patient Charlie (ID: P003) with priority 2.
Current emergency queue (priority, ID, Name):
  Priority 3: Bob (ID: P002)
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code>
```

```

class EmergencyQueue:
    def is_empty(self):
        """
        return len(self._queue) == 0

    def display_queue(self):
        """
        Display the current state of the emergency queue (for debugging).
        Note: This creates a copy to avoid modifying the heap.
        """
        if self.is_empty():
            print("Emergency queue is empty.")
        else:
            temp_queue = self._queue.copy()
            print("Current emergency queue (priority, ID, Name):")
            while temp_queue:
                priority, _, patient_id, patient_name = heapq.heappop(temp_queue)
                print(f"Priority {priority}: {patient_name} (ID: {patient_id})")

# Example usage
if __name__ == "__main__":
    eq = EmergencyQueue()
    eq.add_patient("P001", "Alice", 1) # Critical
    eq.add_patient("P002", "Bob", 3) # Normal
    eq.add_patient("P003", "Charlie", 2) # Urgent
    eq.display_queue()
    eq.treat_next_patient() # Should treat Alice
    eq.treat_next_patient() # Should treat Charlie
    eq.display_queue()

```

Task Description - 7: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.
5. Parking Slot Availability – Track available and occupied slots.

Student Task

- For each feature, select the most appropriate data structure from the list below:

- o Stack
- o Queue
- o Priority Queue
- o Linked List

- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

- A table mapping feature $\rightarrow$ chosen data structure $\rightarrow$ justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```

272 class VehicleRegistry:
299     def remove_vehicle(self, reg_number):
303         """
304         if reg_number in self._registry:
305             del self._registry[reg_number]
306             print(f"Vehicle {reg_number} removed from registry.")
307             return True
308         print(f"Vehicle {reg_number} not found in registry.")
309         return False
310
311     def display_all(self):
312         """
313         Display all registered vehicles (for debugging/admin purposes).
314         """
315         if not self._registry:
316             print("No vehicles registered.")
317         else:
318             print("Registered Vehicles:")
319             for reg, details in self._registry.items():
320                 print(f"    {reg}: {details}")
321
322 # Example usage
323 if __name__ == "__main__":
324     registry = VehicleRegistry()
325     registry.register_vehicle("ABC123", "John Doe", "Toyota Camry", "Blue")
326     registry.register_vehicle("XYZ456", "Jane Smith", "Honda Civic", "Red")
327     print(registry.lookup_vehicle("ABC123")) # Should return details
328     registry.remove_vehicle("XYZ456")
329     registry.display_all()

```


Task Description - 8: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with

AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```
C: > Users > rishi > OneDrive > Desktop > UNI > SEM - VI > AI Assistant Coding > vsodes > Assignment_11(5).py > ...

340 class ShoppingCart:
389     def display_cart(self):
393         if not self.head:
394             print("Cart is empty.")
395             return
396         current = self.head
397         total = 0
398         print("Shopping Cart:")
399         while current:
400             subtotal = current.price * current.quantity
401             total += subtotal

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + - []

PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\rishi\AppData\Local\Programs\Python\Python
n.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/AI Assistant Coding/vsodes/Assignment_11(5).py"
Added Laptop to cart.
Added Mouse to cart.
Updated quantity of Laptop to 2.
Shopping Cart:
Laptop (ID: P001) - Qty: 2, Price: $999.99, Subtotal: $1999.98
Mouse (ID: P002) - Qty: 2, Price: $25.50, Subtotal: $51.00
Total: $2050.98
Removed item P002 from cart.
Shopping Cart:
Laptop (ID: P001) - Qty: 2, Price: $999.99, Subtotal: $1999.98
Total: $1999.98
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code>
```

```
340 class ShoppingCart:
387     | | | print(f"Item {product_id} not found in cart.")
388
389     def display_cart(self):
390         """
391         Display all items in the cart with totals.
392         """
393         if not self.head:
394             print("Cart is empty.")
395             return
396         current = self.head
397         total = 0
398         print("Shopping Cart:")
399         while current:
400             subtotal = current.price * current.quantity
401             total += subtotal
402             print(f" {current.name} (ID: {current.product_id}) - Qty: {current.quantity}, Price: ${current.price}")
403             current = current.next
404         print(f"Total: ${total:.2f}")
405
406 # Example usage
407 if __name__ == "__main__":
408     cart = ShoppingCart()
409     cart.add_item("P001", "Laptop", 999.99, 1)
410     cart.add_item("P002", "Mouse", 25.50, 2)
411     cart.add_item("P001", "Laptop", 999.99, 1) # Update quantity
412     cart.display_cart()
413     cart.remove_item("P002")
414     cart.display_cart()
```